

RESEARCH ARTICLE

AnnotationGym: A Generic Framework for Automatic Source Code Annotation

HAFSAH SHAHZAD¹, (Graduate Student Member, IEEE), AHMED SANAULLAH²,
SANJAY ARORA², ULRICH DREPPER², AND MARTIN C. HERBORDT¹

¹Electrical and Computer Engineering Department, Boston University, Boston, MA 02215, USA

²Red Hat Inc., Raleigh, NC 27601, USA

Corresponding author: Hafsah Shahzad (hshahzad@bu.edu)

This work was supported in part by NSF under Award CCF-1919130; in part by the Grant from Red Hat; and in part by Advanced Micro Devices, Inc (AMD) and Intel, both through donated Field-Programmable Gate Arrays (FPGAs), tools, and Intellectual Property (IP).

ABSTRACT A common approach to code optimization is to insert compiler hints in the source code using annotations. Two major challenges with using annotations effectively are their complexity and lack of portability. This means, first, that significant developer expertise is required, and, second, that the supported annotations, as well as their syntax and use, can vary substantially. Moreover, there is not currently any tool that can output performant annotation-inserted codes for different back-ends. To address these challenges, we present AnnotationGym, an easy-to-use, open-source, generic infrastructure that supplements or replaces the developer in annotating source code. It demonstrates a novel application of AI methods to code annotation. In addition to improving code performance, the flexibility of AnnotationGym enables easy comparisons of performance and optimization strategies among compilers and target architectures and thus provides an extensible platform to facilitate further progress in this field. AnnotationGym automatically extracts structured information about the target code and compiler to generate a list of possible annotations. AI-based optimization algorithms then traverse this space to determine the best set of annotations depending on the developer goals. To demonstrate its effectiveness, we run AnnotationGym on popular, representative workloads from the Polybench suite, as well as targeting various compilers (GCC, AMD HLS, Intel HLS), optimization algorithms (Reinforcement Learning, Bayesian Optimization), and architectures (CPU, FPGA). We also test our approach on FPGA codes derived, e.g., from the Rodinia and OpenDwarfs benchmarks and that are hand-optimized using standard best practices. An interesting finding is that the best overall performance obtained by AnnotationGym was generally with unoptimized codes.

INDEX TERMS Code annotation, reinforcement learning, source-to-source code optimization, compiler hints.

I. INTRODUCTION

A goal of compilation is understanding code patterns and optimizing them for performance. However, a compiler typically makes safe decisions: it is bound by the interpretation of the source code as defined by the programming language standard; even a slight violation is forbidden. Also, a compiler generally cannot predict every detail of code execution; this often results in degraded performance, such as through missed opportunities for parallelization.

The associate editor coordinating the review of this manuscript and approving it for publication was Arman Oshnoei¹.

A. BACKGROUND

Often the programmer has knowledge that can help the compiler better optimize the code: annotations provide one way to convey this information. For instance, annotations can introduce constraints or provide hints about using specific compiler semantics or extensions. Typical hints include use of the memory hierarchy, size of variables, and values of pointers. Thus annotations help the compiler emit faster, smaller and more predictable machine code, by eliminating the need for several runtime checks and removing uncertainty.

While often effective, use of annotations presents a number of problems. First, inserting annotations is an error-prone

and time-consuming process. Second, annotations work best when inserted at specific locations in the code, which are not always obvious. Third, annotation syntax, support, and usage vary widely among compilers: an expert developer for one compiler or target hardware may not be well-versed in another. Fourth, the best positions of the annotations vary with the compiler. For example, the loop unrolling pragma for AMD HLS must be inserted after the *for* loop; in other compilers such as Intel HLS, GCC, Clang, and OpenMP where loop unrolling must be declared before the loop [1], [2], [3], [4]. Finally, even if annotations are inserted correctly, good performance is still not guaranteed. We have observed that incorrect (yet legal) insertion of certain annotations can result in multiple factors worse performance. Infeasible configurations may increase latency costs, cause deadlock, or result in runtime errors. For instance, some annotations are context sensitive and must be annotated with correct variable names and factor values.

These challenges indicate that code annotation, while extremely useful, could benefit from a generic, intelligent, automated toolflow. Our goal is to *extract maximum performance benefit without any additional programming effort, while using state-of-the-art tools to guarantee universal availability*. Ideally, this toolflow should deliver performance for any set of targets (compiler, code, hardware) while preserving semantic and functional correctness. It should also deliver performance benefits not only for a naive user, but also design experts seeking to be more productive or who need an easy way to generate design alternatives.

B. OVERVIEW AND CONTRIBUTIONS

We present AnnotationGym, a framework for automatic code annotation that addresses two clear gaps in existing options. First, they are insufficient in terms of portability, back-end support, automation, and end-to-end use of AI. And second, to the best of our knowledge, there is no such comprehensive, open-source framework that can insert optimal compiler hints and annotations into source codes and that leverages state-of-art AI optimizers for this task. By doing so, AnnotationGym provides the community with a tool that handles code annotations for a broad set of applications, compilers, hardware, developer expertise, and target goals. It is also extensible, allowing integration of new techniques.

An overview of AnnotationGym operation is as follows. Given a set of objectives, the tool automatically determines a list of all possible annotations that can be applied, regardless of their validity. It then pre-processes the list and prunes it to remove any illegal annotations. Finally, an AI-based optimization algorithm is used to search the list and determine the best combination based on a compiler specific reward function. We note that such AI techniques have already proven invaluable and shown to outperform human experts in other contexts [5], [6], [7]. In this work, we ask: Can these optimization algorithms also efficiently annotate source codes?

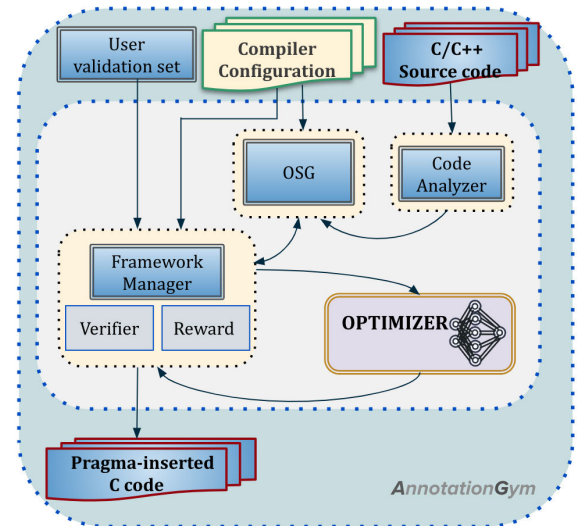


FIGURE 1. Overview of the AnnotationGym framework.

AnnotationGym is designed to ensure functional correctness to ensure broad applicability. For CPU targets, the framework uses a validation step to discard invalid annotations. The framework itself is designed in a highly modular fashion with well-defined interfaces. This allows nearly all aspects of the system—e.g., code, compiler, optimizer, and annotation generation—to be independently selected and customized. Figure 1 gives the overall workflow.

We demonstrate the performance benefits of this approach through a number of common benchmarks (derived, e.g., from Rodinia, and Polybench [8], [9], [10]); for two contrasting hardware types, CPU and FPGA; and for different compilers, including GCC, AMD HLS, and Intel HLS [2], [3]. For FPGA target architectures, we further demonstrate how AnnotationGym can optimize codes by comparing with hand optimized versions. The results lead to observations about the importance of exposing degrees of freedom to increase the effectiveness of automated tools.

For the same target hardware and application, AnnotationGym allows the user to compare optimization strategies preferred by different compilers while optimizing for the same goal. For example, we compare Intel HLS and AMD HLS post Place-and-Route metrics of frequency and resource usage for baseline versus AnnotationGym optimized codes. We note further that if the developer goal is latency reduction, HLS compilers may achieve this in a number of different ways, e.g., (i) by creating efficient pipelines thereby increasing frequency and decreasing resource usage; (ii) or by decreasing both the frequency and resource usage yet extracting latency advantage through runtime reductions; (iii) or by decreasing both runtime and latency by increasing computations and hence the resource usage.

Specifically, this work makes the following contributions:

- An automated and portable approach to source code annotation that substantially reduces requirements for developer effort and expertise;

- An extensible framework with well-defined interfaces that achieves generality across compilers, benchmarks, computational hardware, and optimization algorithms;
- A validation methodology incorporated within the framework to verify the functional and semantic correctness of the generated annotation sets;
- Evaluations across multiple representative benchmarks, compilers, computational platforms, and optimization strategies that show the effectiveness of this approach. For the evaluated benchmarks common to the two target hardware, we obtain, on average, a $57\times$ performance improvement for AMD HLS, $5.6\times$ for Intel HLS, and $1.8\times$ for GCC. This benefit persists when tested against otherwise optimized baselines [11].
- Performance improvements over baseline high level language code can be difficult to interpret especially for FPGA workloads. We have therefore hand-optimized common FPGA workloads codes for FPGA backends using a procedure based on best practices [10] and applied AnnotationGym. Our results show that it is possible to achieve further improvements over the best hand optimized codes: a $32.3\times$ performance improvement for AMD HLS and $3.1\times$ for Intel HLS [12].

The significance is as follows: (i) The first work to apply ML and learning based algorithms to automating the insertion of annotations; (ii) Provides an end-to-end, effective tool for insertion of annotations that gets excellent results using standard evaluation methods and includes an automated validation method; (iii) Provides benefits across the spectrum of hardware design experience. Less experienced developers find potentially unfamiliar optimization strategies are automatically explored; experienced developers can port known hardware optimization strategies without going through massive documentation to determine specific annotations.

The rest of this paper is organized as follows. Section II discusses related work in automated code annotation. Section V presents the proposed framework. Section VI describes the evaluation methods. Section VII evaluates the approach. Finally, Section VIII gives a conclusion and future directions.

II. LITERATURE REVIEW

While automatic code optimization as part of the compiler is an essential component in creating performant code [13], it has often been found that the effectiveness of these optimizations is greatly influenced by the characteristics of the code source [14], [15]. This has led to a number of studies exploring modification of source code with respect to optimizer performance [16], [17] for better automatic optimization. These methods include best programming practices [10], [12] and automatic source-to-source code transformations.

Performance in this context can refer to decreasing execution time, generating smaller binary code or reducing energy usage [5]. A key characteristic of this area of research

is that static/generic compiler strategies are suboptimal for diverse workloads. Optimizations applied such as compiler flags or annotations need to be empirically guided and adapted to the program structures and target architectures [11]. A primary contribution of AnnotationGym is that it effectively supports target architectures with extremely different code characteristics, i.e., both CPU programs that direct sequential execution of instructions and FPGA programs that describe circuits. We now describe prior work for both, concentrating on the methods based on code annotation.

For FPGA as target architecture, the community has suggested pre-processing source code manually using FPGA vendor or universally accepted best practices [10], [18]. However the limitation with all such approaches is that significant expertise is needed both in the target compiler and also the target hardware. Manual efforts are also not a feasible approach in terms of time and effort. In addition, such approaches are not portable: the source code needs to be significantly modified and can no longer be specific to a compiler regardless of target architecture.

An alternative approach to manual source code restructuring is automated source code annotation. This is addressed in this work with emphasis on portability, programmability, flexibility, extensibility, modularity, generality and performance. This approach is portable since the source code is not modified. The framework outputs a pragma-inserted version of the source code according to the user configurations. Some previous work has looked at source code annotation, particularly for CPU codes. However, the work is limited to certain annotation libraries such as OpenMP and specific aspects of the code. For FPGAs, the community has largely explored design space exploration of pragma parameters using heuristic based approaches focusing on dataflow or loops and for specific runtime libraries and domain space languages. AnnotationGym addresses limitations of previous work both for CPU and FPGA code annotations. The details are discussed below.

A. CPU AND GPU CODE ANNOTATION

Traditionally annotation libraries have been used to grant parallel semantics to code syntax that is originally written to run sequentially; examples include OpenMP, OpenACC, OpenHMPP, and OpenSs. A number of existing tools [16], [19], [20], [22], [23], [24] target automatically inserting OpenMP pragmas into source code. These tools have their own caveats: the output generated varies in format, i.e., source-to-source versus binary; pointer aliasing hinders parallelization; and additional checks are needed to ensure that dependencies are avoided. Loop parallelization strategies also differ: Cetus [19] deals with multi-core parallelization, while DawnCC [20] targets GPU threading. A major drawback of all these works is lack of portability: they are targeted towards specific compilers and support input codes in specific formats. Also, only a subset of available annotations is generally supported.

TABLE 1. Key feature comparison between AnnotationGym and prior annotation frameworks.

Feature	AnnotationGym	Cetus [19]	DawnCC [20]	IronMan [21]	Other FPGA DSE Tools
Target Architecture	CPU, FPGA	CPU	CPU, GPU	FPGA	FPGA
Annotation Scope	Loops, flags, pragmas, interfaces, pipelines, resources	multi-core parallelization	GPU threading	Resource allocation (DSP/LUT)	Loop unrolling, pipelining, arrays
Automation Type	Full pipeline (pragma insertion)	Auto OpenMP insertion	Auto OpenMP/OpenACC insertion	Resource tuning only	Heuristic-based tuning
AI Technique	RL (PPO), BO	None	None	RL (Actor-Critic, REINFORCE)	None / Limited
Portability	High	Low	Low	Low	Low
Remarks	Broad, extensible annotation support across compilers	OpenMP only; CPU-focused	Compiler/pragmatic-specific for CPU/GPU	Resource tuning only; require substantial dataset	Tool-specific; Limited to certain FPGA tools, no automated framework, limited AI scope

Other frameworks explore the space of loop transformations, e.g., in Clang by employing LLVM’s polyhedral loop nest optimizer (Polly [4], [25], [26]), and optimizing loop vectorization parameters inside Clang’s intrinsic pragmas [6]. AnnotationGym explores a broader set of annotations not tapped by existing work, e.g., by providing options to turn on certain compiler optimization flags [27] and by supporting instruction set extensions [1].

B. FPGA CODE ANNOTATION

Much research in the space of optimizations for FPGAs, has focused on design space exploration [28], [29], [30], [31], [32], [33]. Such traditional heuristic based approaches such as simulated annealing and genetic search are not efficient in searching the design space for HLS codes. There is a need for highly efficient learner to analyze the search space and problem explored. In our work we use a data-driven approach utilizing a Reinforcement Learning (RL) agent. We strongly believe this is a promising approach for navigating future HLS landscape. IronMan [21] is one work in this space that utilizes RL. However they have explored use of RL to identify optimal resource allocations between DSP and LUT. We utilize the power of RL in not only identifying resource allocation, but also optimizing loop transformations, identifying appropriate interfaces specific to the HLS tool, and optimizing the pipelines. In addition, IronMan uses RL learners like Actor-Critic and REINFORCE. We explore Proximal Policy Optimizer (PPO), which is state-of-the-art and widely considered one of the best RL algorithms available [34]. A limitation of policy gradient methods, including one used by IronMan, is that they require a substantial dataset. PPO is considerably more sample efficient, and also robust and stable, making it a good choice to learn across variety of HLS workloads. We also extend prior work by supporting automatic insertion of annotations into the source code, thereby creating an end-to-end flow. AnnotationGym leverages AI to automate the insertion of annotations. It also demonstrates the applicability of RL: this is significant since RL reduces the search cost by using past data to drive optimization decisions [5].

Several studies identify regions of interest, or predict optimal factors for loop-based HLS directives, such as unrolling or pipelining. A major contribution of this work lies in creating a massive search space for multiple state-of-the-art HLS compilers and deriving performance also by leveraging HLS directives for optimizing functions and arrays. For example, authors in [35] have explored code annotations for data-flow programming in streaming applications that have multiple kernels within a single code. The source-to-source compiler, SpecHLS [36] targets a subset of HLS kernels that can benefit from speculative pipelining strategies. Reference [37] present an optimizer for design space exploration within the Merlin compiler. Others, such as AutoScaleDSE [38], use their own quality-of-results (QoR) estimator to provide performance estimates and optimize for loop tiling and array partitioning.

Other contributions of AnnotationGym are as follows. First, AnnotationGym allows HLS code annotation for AMD and Intel HLS and allows incorporation of multiple domain specific languages and compiler tools [39], [40] to enable exploration of the optimization design space using learner-feedback-based RL agents. Second, to improve training accuracy, AnnotationGym evaluates all performance results using downstream HLS tools. Third, it also supports verification. The optimized codes are tested not only to ensure functional correctness, but also implemented on the target hardware to evaluate run time, resource usage, and frequency after place-and-route. Finally, it demonstrates how annotated baseline codes compare with respect to their hand-optimized versions. Table 1 summarizes the key distinctions between AnnotationGym and prior annotation tools and frameworks.

III. ARTIFICIAL INTELLIGENCE FOR CODE ANNOTATION

Developing effective source-to-source translation process requires an intelligent strategy. This work explores utilizing artificial intelligence (AI) in driving optimal choices. Here we discuss background of the approach used and introduce Markov Decision Process and its implementation as RL.

A. MARKOV DECISION PROCESS

Markov Decision Process (MDP) is a stochastic process that models decision-making in fully observable environments. In a typical MDP, an *agent* is the *learner* that learns through its decisions and interactions. An *environment* is a demonstration of the *problem* in which the agent interacts. MDPs are defined as a tuple:

$$\langle S, A, P, R, \gamma \rangle$$

where:

- S is the set of possible situations, configurations, and positions of the agent within the environment;
- A is the set of possible actions/decisions that the agent makes and through which it perturbs the environment in order to learn;
- $P(s'|s, a)$ is the state transition probability function. It gives the likelihood that the agent transitions from state s to another state, s' after taking an action a ;
- $R(s, a)$ is the reward function, giving the immediate reward that the agent gets after performing a certain action a in some state s of the environment; and
- $\gamma \in [0, 1]$ is the discount factor. It is a means to qualify how much importance has to be placed on immediate versus future rewards.

MDPs are based on the Markov Property: future states depend only on the current state and action (and not on the past sequence of states). Solving an MDP essentially means finding a policy or strategy that chooses an action based on the current state and optimizes for the overall reward/outcome.

B. REINFORCEMENT LEARNING AS AN MDP

Our approach for solving MDPs is to use RL, which obeys the Markov property in that the current state is defined to capture information of the past states; i.e., only the current state and action are required to determine the future state. There are two major components to the overall framework:

- *Environment*: the problem that we are trying to learn to solve
- *Agent*: used to perturb the environment and learn based on feedback.

The ultimate goal in RL is to compute a mapping between the environment and actions: a policy $\pi(a|s)$ that is maximizing the expected cumulative reward. This policy is defined as a probability distribution over actions given a state and is given as:

$$\mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right]$$

where s_t and a_t denote the state and action at time step t , respectively. In RL, the agent learns this policy using trial-and-error mechanism, interacting with the environment and receiving a reward (feedback) in return. This is contrary to supervised learning where labeled data is required in order for the model to learn. Further details of how the RL is set up for code annotation is discussed in Section VI-C1.

C. PROXIMAL POLICY OPTIMIZATION (PPO)

Proximal Policy Optimization (PPO) is one of the most popular RL algorithm and is often favored for its training stability and efficiency. Unlike other algorithms, PPO keeps the policy update simple. It optimizes the policy by maintaining a balance between exploration (trying new actions) and exploitation (utilizing actions that have shown previously to result in high rewards). In addition, it is robust to hyperparameter settings and handles large state spaces effectively [41]. As discussed in Section II-B, we experimented with several state-of-the-art algorithms and found PPO to be the most stable. Moreover, PPO is one of the most computationally efficient choices: while other methods mostly involve second-order optimization techniques, PPO is a first-order optimization algorithm [42]. These benefits make PPO particularly effective for high-dimensional problems such as code annotation.

PPO uses a Fully Connected Network (FCNet) with [256,256] hidden layers and tanh activation function to implement its policy and value functions. The policy is updated using PPO's *clipped* objective function. Clipping restricts the size of policy updates allowing the policy to update in a stable manner without collapsing. The objective function to be maximized is given by:

Algorithm 1 PPO Algorithm

```

Initialize policy  $\pi_\theta$  and value function  $V_\theta$ 
for each iteration do
    Collect trajectories  $\{s_t, a_t, r_t\}$  using  $\pi_\theta$ 
    Calculate the advantage function  $A_t$ 
    Update policy by maximizing  $L^{CLIP}(\theta)$ 
    Update value function by minimizing  $L_V(\theta)$ 
end for

```

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t) \right]$$

where: - $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ is the probability ratio of the new and old policy for action a_t at state s_t , - A_t is the advantage function at time step t , which estimates how much better or worse an action was compared to the baseline. A_t is typically computed as:

$$A_t = \delta_t + (\gamma\lambda)\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \dots$$

where $\delta_t = R_t - V(s_t)$ is the temporal difference error, and γ and λ are hyper-parameters controlling the balance between bias and variance in the advantage estimate. ϵ is another hyperparameter controlling how much the policy is allowed to change.

After each update, the parameters of the policy are adjusted using stochastic gradient descent (SGD) to maximize the objective $L^{CLIP}(\theta)$. The update rule is:

$$\theta_{new} = \theta_{old} + \alpha \nabla_\theta L^{CLIP}(\theta)$$

where α is the learning rate.

In addition to updating the policy, PPO also updates the value function $V_\theta(s_t)$, which estimates the expected return from a state. The value function is updated by minimizing the following loss:

$$L_V(\theta) = \frac{1}{2} \mathbb{E}_t \left[(V_\theta(s_t) - R_t)^2 \right]$$

where R_t is the return at time t .

The final PPO algorithm alternates between collecting trajectories using the current policy, calculating advantages, and updating both the policy and the value function. The algorithm proceeds as follows:

IV. BACKGROUND FOR HIGH LEVEL SYNTHESIS

In this section we look at the programmability model for traditional HLS tools and the available knobs for users to pre-process their codes using directives (pragmas and annotations). We find that the complexity motivates the RL assisted framework. We then discuss an alternate pre-processing approach, namely, source code rewriting, and show that it can be combined with RL.

A. PROGRAMMABILITY MODEL FOR HLS TOOLS

While HLS tools have raised the abstraction level for programming FPGAs, extracting performance from the input code is still a challenge. HLL codes are usually sequential and typically written by developers with little knowledge of FPGAs' essential characteristics. Most HLS tools improve performance at their back-end by leveraging data-level parallelism using dependency analysis. Much parallelizability, however, is hard to predict. For this, tools such as AMD HLS, Intel HLS, and Microchip's SmartHLS [2], [3], [43] allow users to insert annotations to highlight explicit parallelism, pipelining opportunities, and optimal memory interfacing.

The task of pragma insertion is notoriously demanding. Tables 4 and 5 in Section VI-B list some available annotations for HLS tools that have been explored in this work. Since applications investigated are single kernels, the scope of this study does not include task level pipelining and structure packing. In HLS the constructs that have the highest impact on the final RTL micro-architecture are *functions*, *loops*, and *arrays*. Typically, an HLS tool first converts each function into a specific hardware component. For a given code, its ports and *functions* can be either inlined or not. Inlining is especially useful for functions that are small and rarely called. These can be inlined into a larger caller function, allowing operations within the inlined function to be shared and optimized effectively. However, inlining can also worsen performance, specifically when the inlined function needs to be called multiple times within the parent function [2].

Next, *Loops* can be unrolled completely or only partially. Unrolling introduces hardware duplication. The amount of unrolling, however, is constrained by the memory bandwidth. *Loops* can also be pipelined with different initiation intervals. But an inter-iteration loop-carried dependency will cause loop pipelining to fail.

Finally, *Arrays* can be mapped to registers or memories of different types. This is critical since interface contention—i.e., a RAM that allows fewer reads/writes than accesses in the same iteration—will cause a bottleneck. Deciding on the optimal number of concurrent ports for memory accesses is important so that if multiple arrays are feeding data to a compute task, these arrays can be mapped to different interface ports so as to access the data in parallel. Also, the data must be stored in different memory banks or the accesses will become serialized. Declaring appropriate interface pragmas allows users to resolve such hardware contentions.

For designs with large numbers of loops, arrays, and functions, it is impossible to systematically explore all of the different design-space combinations. To approach an optimal solution, an expert who can adapt rapidly is required. In this work, we have leveraged the particular capabilities of AI so that the expertise and adaptability come from RL.

B. BEST PRACTICES OPTIMIZATIONS OF BASELINE CODES

In prior work using source code transformation [10], [17], [18], researchers benchmarked common FPGA workflows by exploiting best practices for manual source code reconstruction. These best practices come mainly from (i) vendor-specific reference manuals; (ii) universally applied code optimization strategies such as use of loop unrolling and loop fusion; and (iii) FPGA-specific good practices, such as allowing concurrent memory accesses, using memory bandwidth effectively, and efficient resource binding. Sometimes best practices are not compiler agnostic; programmer understanding of HLS compiler behavior and FPGA architecture remains essential. For instance, published best practices sometimes result in reduced performance [10].

In some cases *source code reconstruction offers benefits not possible with annotations alone*. This observation illustrates limitations of current HLS compilers: not all performance can be extracted using just compiler hints; in some cases programmer involvement is still required. Some examples include:

- reordering of loops for spatiality and for exploiting input caching;
- loop tiling to remove loop carried dependencies;
- rewriting code to remove overlapping array accesses;
- fusing multiple loops to achieve runtime reduction;
- extracting task level parallelism by implementing independent tasks, such as kernels, and connecting them using channels;
- restructuring code to remove conditional accesses and if-else conditions that can create bottlenecks for parallel execution; and
- since some HLS compilers cannot estimate latency if a while loop is present, converting them into for loops.

Accordingly, evaluations of HLS enhancements, such as those proposed in this study, *should have multiple baselines*:

TABLE 2. Summary of code versions and optimizations applied.

Version	Optimization
1	CPU-like single kernel C code
2	Implement task parallel computations in separate kernels connected using channels
	Apply optimizations such as loop unrolling, minimizing variable declaration using temps
	Using constants for problem sizes and data values to reduce off-chip memory access
3	Coalesce memory operations
	Implement all optimizations proposed for version 2 within a single kernel instead
4	Reduce array sizes to infer pipeline registers as registers, instead of BRAMs
5	Perform computations in detail, using temporary variables to store intermediate results
6	Use predication instead of conditional branch statements



FIGURE 2. Code Analyzer: Source code is parsed and analyzed to create a structured representation.

the original code, but also code to which best practices have been applied. The latter is essential since it represents versions of the code that are in states that are (apparently) not accessible to those HLS compilers. As a result our evaluations are with respect to both baseline code (version 1) and code that has undergone programmer-based reconstruction (as in Table 2). As we see in Section VII, however, it turns out that AnnotationGym on *unoptimized* code most often gives the best results.

V. DESIGN OF THE ANNOTATIONGYM FRAMEWORK

Among the design requirements for AnnotationGym is that it be extensible so that new features can be added; modular, so that different blocks can be customized separately based on user requirements; and that it support validation and verification. Together, these allow AnnotationGym to be used for virtually any combination of workload, compiler, and design space exploration algorithm. A conceptual map of the framework is given in Fig. 1.

A. CODE ANALYZER

The target source code (C/C++) is first parsed and analyzed to extract its annotation potential – i.e., the unique ways that the code can be annotated – and also details necessary for further processing. The latter include: (i) locations in the source code where annotations applicable to the entire code (global

```

"COMPILE_COMMAND": "i++ -d.e.c -march=Stratix10 -o output --simulator none --fpga-only --clock
300MHz",
"VALIDATE_COMMAND": "i++ -march=Stratix10 -ghdl main.c -o main && ./main",
"REWARD_EVALUATE": ":",
"LATENCY": ":",
"ANNOTATION_TYPE": {
  "builtins for pointer interfaces": [
    {"ID": "Pointer_Interface", "Name": "@4"},
    {"ID": "Pointer_Interface", "Name": "@4_restrict"},
    {"ID": "Pointer_Interface", "Name": "i::hmm:host @5, i::h::aspace1d, i::h::awidthd4*sizeof( @5 )",
      i::h::widthd8*sizeof( @5 ), i::h::alignd8*sizeof( @5 ) > >"},
    {"ID": "Pointer_Interface", "Name": "i::hmm:host @5, i::h::aspace1d, i::h::awidthd4*sizeof( @5 )",
      i::h::widthd8*sizeof( @5 ) > >"},
    {"ID": "Pointer_Interface", "Name": "i::hmm:host @5, i::h::aspace2d, i::h::widthd8*sizeof( @5 )",
      i::h::latencyd0, i::h::maxburstd0, i::h::waitrequest(true) > >"},
    {"ID": "Pointer_Interface", "Name": "i::hmm:host @5, i::h::aspace2d, i::h::widthd8*sizeof( @5 )",
      i::h::latencyd0, i::h::maxburstd0, i::h::alignd8*sizeof( @5 ), i::h::waitrequest(true) > >"},
    {"ID": "Pointer_Interface", "Name": "i::h::avalon_agent_memory_argument( @6 *sizeof( @5 ) @4 )",
      i::h::widthd8*sizeof( @5 ) > >"},
    {"ID": "Pointer_Interface", "Name": "i::h::avalon_agent_memory_argument( @6 *sizeof( @5 ) ) his_numbanks(4)
      his_bankwidth(4) @4"},
    {"ID": "Pointer_Interface", "Name": "i::h::avalon_agent_memory_argument( @6 *sizeof( @5 ) ) his_numbanks(1)
      his_bankwidth(16) @4"},
    {"ID": "Pointer_Interface", "Name": "i::h::doublepump his_avalon_agent_memory_argument( @6 *sizeof( @5 ) )
      volatile @4"},
    {"ID": "Pointer_Interface", "Name": "i::h::numbanks(1) his_singlepump his_avalon_agent_memory_argument( @6
      *sizeof( @5 ) ) his_readwrite_mode( "writely") volatile @4"},
    {"ID": "Pointer_Interface", "Name": "i::h::avalon_agent_register_argument @4"}],
    "builtins for constants": [{"ID": "Constant_Interface", "Name": "@4"},
      {"ID": "Constant_Interface", "Name": "his_stable_argument @4"},
      {"ID": "Constant_Interface", "Name": "his_avalon_agent_register_argument @4"}],
    "pragmas before for loop": [{"ID": "Loops", "Name": ""}, {"ID": "Loops", "Name": "#pragma unroll
      62"}, {"ID": "Loops", "Name": "#pragma max_concurrency 1"}, {"ID": "Loops", "Name": "#pragma
      max_interleaving 1"}, {"ID": "Loops", "Name": "#pragma max_interleaving
      9"}, {"ID": "Loops", "Name": "#pragma disable_loop_pipelining"}, {"ID": "Loops", "Name": "#pragma ii
      1"}, {"ID": "Loops", "Name": "#pragma lvdpr"}, {"ID": "Loops", "Name": "#pragma loop_coalesce"},
      "Global": [{"ID": "Global", "Name": "#pragma clang fp contract(fast)", {"ID": "Global", "Name": "#pragma
      clang fp contract(off)", {"ID": "Global", "Name": "#pragma clang fp contract(on)", {"ID": "Global",
      "Name": "#pragma clang fp reassociate(on)", {"ID": "Global", "Name": "#pragma clang fp
      reassociate(off)", {"ID": "Global", "Name": "his_disable_component_pipelining"}, {"ID": "Global",
      "Name": "his_max_concurrency(0)", {"ID": "Global", "Name": "his_use_stall_enable_clusters"}]]}

```

FIGURE 3. Example: Compiler Configuration Script for Intel HLS.

```
"COMPILE_COMMAND": "vitis_hls -f run_hls.tcl",  
"VALIDATE_COMMAND": "vitis_hls -f validate.tcl",  
"REWARD_COMMAND": {  
    "LATENCY": "echo -e '\\import json\\ntry:\\nwith open('code_proj/solution1/solution1_data.json') as f:\\n\\nresults = json.load(f)\\n\\nmetric=results['ModuleInfo']['Metrics'][results['Top']]['Latency']['LatencyWorst']\\nexcept Exception as e:\\nrewards['REWARD':metric]\\nwith open('reward','w') as f:\\n\\njson.dump(reward,f)'\\n'\\n python )" }  
  
    "Annotation Type": {  
        "builtins after function declaration": {  
            {"ID": "Function", "Name": "#pragma HLS INLINE off", "Count_of_variables":0},  
            {"ID": "Function", "Name": "#pragma HLS INLINE", "Count_of_variables":0},  
            {"ID": "Function", "Name": "#pragma HLS INTERFACE mode=s_axilite port=return", "Count_of_variables":0},  
            {"ID": "Function", "Name": "#pragma HLS INTERFACE mode=s_axilite port=return bundle=AXILITEs", "Count_of_variables":0},  
            {"ID": "Function", "Name": "#pragma HLS INTERFACE m_axi port=@2 offset=off", "Count_of_variables":1},  
            {"ID": "Function", "Name": "#pragma HLS INTERFACE m_axi port=@2 offset=direct", "Count_of_variables":1},  
            {"ID": "Function", "Name": "#pragma HLS INTERFACE m_axi port=@2 offset=slave bundle=@3", "Count_of_variables":2},  
            {"ID": "Function", "Name": "#pragma HLS ARRAY_PARTITION variable=@2 type=cyclic factor=8 dim=1", "Count_of_variables":1},  
            {"ID": "Function", "Name": "#pragma HLS ARRAY_PARTITION variable=@2 type=complete dim=0", "Count_of_variables":1},  
            {"ID": "Function", "Name": "#pragma HLS ARRAY_PARTITION variable=@2 type=complete dim=1", "Count_of_variables":1},  
            {"ID": "Function", "Name": "#pragma HLS ARRAY_PARTITION variable=@2 type=block factor=@4 dim=1", "Count_of_variables":1},  
  
            "pragmas after for loop": [{"ID": "Loops", "Name": "#pragma HLS UNROLL"}, {"ID": "Loops", "Name": "#pragma HLS UNROLL factor=@5"}, {"ID": "Loops", "Name": "#pragma HLS PIPELINE"}, {"ID": "Loops", "Name": "#pragma HLS PIPELINE rewind"}, {"ID": "Loops", "Name": "#pragma HLS PIPELINE enable_flush"}, {"ID": "Loops", "Name": "#pragma HLS PIPELINE II=1"}],  
  
            "global": [{"ID": "Global", "Name": "I", "Options": [ ]}]}
```

FIGURE 4. Example: Compiler Configuration Script for AMD HLS.

scope) can be inserted; (ii) functions in the code, including their declaration points, lists of function arguments and local variables, and lists of loops; (iii) arguments and variables within each function, including names and types (e.g., pointer and read-only); and (iv) loops within each function, including declaration points, unique ID, structural details (loop nesting levels), and loop iterator information (name and type). Once the required details have been extracted, a structured representation of the code is generated. Fig. 2 shows the code analyzer output for `gemm` (general matrix multiply).

The output from the Code Analyzer is same regardless of the compiler. This output is used by the Optimization Space Generator (OSG) to determine where annotations can be inserted, and provides context for annotations that target specific code variables.

B. COMPILER CONFIGURATIONS

Compiler configurations are specific to a compiler and provide the information needed by the framework for generating annotations, compiling code, performance measurement, and validating results. As an example, the compiler configuration script for Intel HLS is shown in Fig. 3. Annotations for loops, data interfaces, and functions as a whole (global) are registered separately. Similar scripts are also generated for other compilers such as AMD HLS and GCC. Figure 4 shows example configuration script for AMD HLS. If a user wants to register an annotation or directive into the framework, they can simply append that information into the compiler configuration script for the specific compiler. The configuration file provides details such as:

Compiler specific commands are for i) compiling and running, ii) profiling and evaluating with respect to the target metric, and iii) validating the annotated source code. As a default, the evaluation metric is configured as latency in cycles. Users can also optimize for other metrics such as area, or a combination thereof. These commands can be arbitrarily complex; and any additional files needed to support the execution of these commands can be linked during compile and/or run-time. Thus virtually any combination of tooling can be interfaced within the framework.

A list of annotations supported by the compiler is organized into three categories: Global, Function, and Loop. Global annotations include high level optimization options and target hardware specific options. Function annotations typically convey information about function variables such as memory type, alignment, and memory arguments. These attributes can also be specific to the target hardware such as array partitioning, type of memory protocol, interface bundles, latency, and, for HLS [2], alignment byte size. Loop annotations are applicable to individual loops, e.g., unroll, flatten, merge, and pipeline. For some annotations, context from the code is needed to properly apply the annotations, e.g., variable names, types, and factor values.

C. OPTIMIZATION SPACE GENERATOR

The Optimization Space Generator (OSG) abstracts the code and compiler details from the rest of the framework, specifically, through the following.

(i) **Annotation Generation:** The OSG takes its input from the code analyzer and compiler configuration and outputs a list of possible annotations and their corresponding insertion points for the specific source code. This is done by traversing the different IDs assigned to global, function, and loop-level annotations. It then shortlists the names and types of variables in each function and generates annotations with details in the appropriate namespaces. These details include variable names, types, and sizes; interface-specific details such as bundle sizes; and optimal factors for unrolling.

(ii) **Code Annotation:** The Framework Manager provides the OSG with a list of annotations to be applied. The OSG then parses the source code and inserts each annotation at

the appropriate location in source code. It then outputs this annotated version of the source code.

D. OPTIMIZER

AnnotationGym is not tied to any specific learning algorithm or optimization strategy. Instead it allows the user to choose the optimization technique best suited to the problem. The Optimizer has the following inputs:

(i) **Number of exploration rounds** gives the number of source code annotations that will be tested by the Optimizer to reach a final best_case. Typically, the greater the number of trials, the more likely the Optimizer is to learn a best set of annotations. However, this also increases the training time and must itself be optimized according to the application and the learning rate of the Optimizer algorithm.

(ii) **Optimization Space** is represented as a list of all annotations, as well as a range of values that can be selected for each annotation. In the simplest case, possible values are 0 (apply) or 1 (skip). The range for each annotation and the significance of each value in the range are handled outside the Optimizer. This allows greater control over the design space exploration, e.g., adding biases, without modifying or restricting the Optimizer itself. The Optimizer is also provided with a reference to the reward function, which takes as input a list of annotations and their corresponding values as determined by the algorithm.

E. FRAMEWORK MANAGER

The Framework Manager coordinates the OSG, the Optimizer, and any calls to tools outside the framework (such as compilers and profilers). It begins by initializing the OSG object. The OSG object returns a list of all possible annotations. The Framework Manager uses this list, as well as any user- or compiler-specific constraints, to create an effective action space for the Optimizer. Next, this space is passed to the Optimizer along with the user-specified number of rounds of optimization exploration and a reference to the reward function. The Framework Manager receives the output of the Optimizer and the list of Optimizer-suggested annotations, together with their insertion points is passed to the OSG.

The OSG, in turn, annotates the code. Using information given by the compiler configuration, the Framework Manager can launch compilation, profiling, and validation runs for this annotated code. To support these operations, the Framework Manager contains two modules, which are described next.

1) VERIFIER

The Verifier has two primary responsibilities.

(i) **Pruning: This step applies to CPU codes only.** The list of annotations from the OSG can be pruned to remove illegal annotations and reduce the search space. Currently, pruning is done by applying each annotation individually to the source code and validating the resulting code using the compile command of the specific compiler. Thus pruning is an optional step that can improve results for a subset

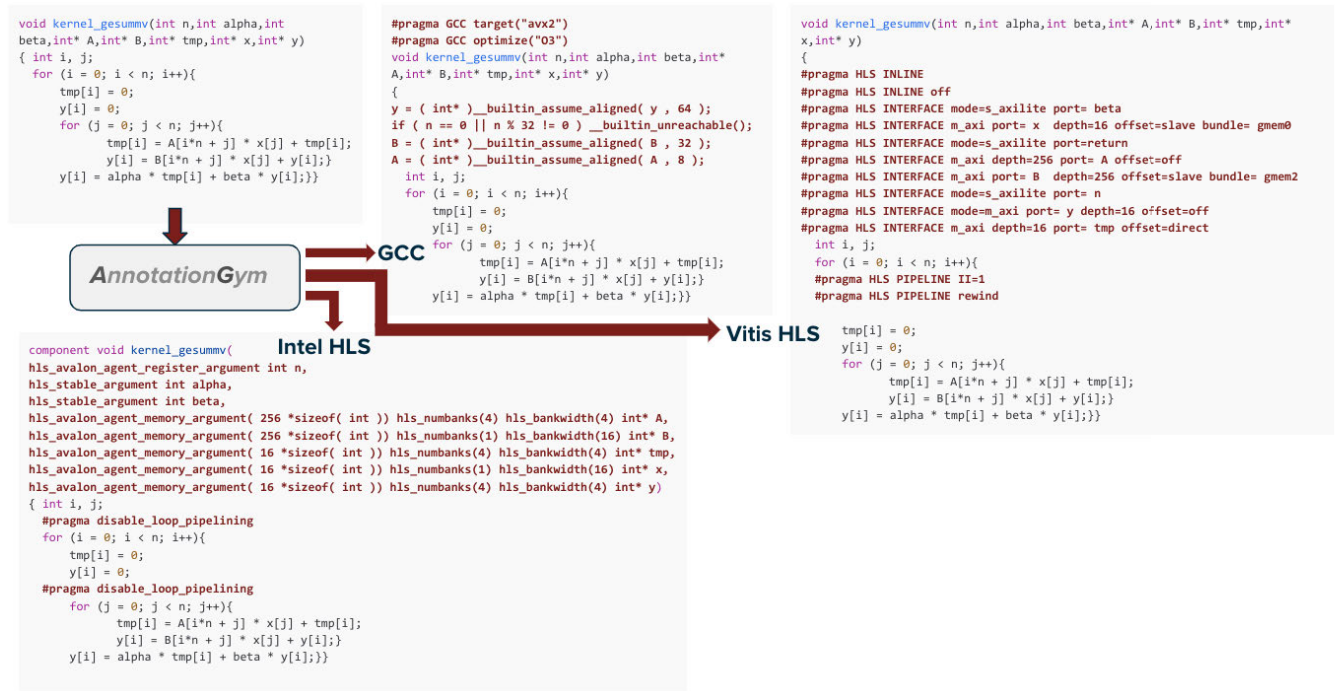


FIGURE 5. Example of automatically pragma-injected code output by AnnotationGym for GCC, AMD HLS, and Intel HLS.

of compiler-code pairings. It is especially useful for code annotations within GCC and LLVM when alignment-specific information is provided by the *User Validation Set* (as seen in Fig. 1) and memory locations of specific arrays overlap. In this case applying the pragma `__builtin_assume_aligned` might produce wrong results and hence will be pruned.

(ii) **Validation:** To validate an annotated code, the framework evaluates correctness using a user-specified validation set. For HLS compilers, this is a typical testbench file used to ensure that the RTL generated is functionally equivalent to the source code. For CPU compilers, this step ensures that the output is same as for the unoptimized source code. To improve turnaround time, validation is not done at every exploration round, but only for annotated code whose performance exceeds the current best result. If validation succeeds, the best case result is updated. Otherwise, the set of annotations is discarded; this action is indicated to the Reward Function and the reward is updated accordingly. Fig. 5 shows an example of optimal annotated code output for three compilers.

2) REWARD FUNCTION

The Reward Function is responsible for running the compilation and profiling steps on the OSG code output; determining the resulting reward values; and modifying the reward signal to more quickly guide an agent toward desired behavior. The reward function also keeps track of i) the best reward and its corresponding annotations and ii) the mapping of the Optimizer action space to the actual design space.

VI. EVALUATION METHODS

A. BENCHMARKS

The framework is evaluated using the Polybench/C benchmark [9] (version 4.2.1). This benchmark is chosen because: (i) the majority of the work in code optimization (CPU,FPGA) has been evaluated using this [6], [21], [37], [38], [40], [45], [46], [47], [48], [49], [50], [51] and (ii) it contains a wide range of practical applications and so is appropriate for general evaluation. For FPGA backends, we have also evaluated other popular FPGA workloads such as (1) Needleman Wunsch (NW) (16K×16K) (2) Particle Mesh Ewald (PME) (100,000 particles and 32 × 32 × 32 grid, and (3) Sparse Linear Algebra (SpMV) (1K×1K matrix) (4) Fast Fourier Transform (FFT)(64 point Radix-2 1D FFT, 8192 Vectors) (5) Range Limited Molecular Dynamics (RL) (180 particles per cell, 15% pass rate) (6) Matrix Multiplication (MMM)(1K x 1K Matrix, Single Precision) (7) Cyclic Redundancy Check (CRC)(100MB CRC32) These are derived from mainstream FPGA benchmarks including Rodinia [8] and OpenDwarfs [52], [53].

We have edited the benchmarks such that all arrays are referenced as pointers. This is done to bring the code into more common representation. Some languages (as opposed to libraries) do not have an array data type. Pointers allow for a more general representation of the data. However, languages sometimes do not convey much knowledge of the extent of the pointer. Annotations provide one way to guide compilers about overlapping memory and alignment of the pointers.

TABLE 3. GCC annotations explored in this work [44].

Builtin	Additional options	Additional details
if (n==0) n%x !=0) __builtin_unreachable	x=8,15,32; n=loop limit	tells compiler that size of loop limit will always be nonzero or multiple of x
__builtin_assume(block_size < n && n% block_size==0)	used for nested loops	n=loop limit
var_x=(var_x_type*)__builtin_assume_aligned(var_x,n)	n=8,32,64; type=int/double/float	tells compiler to assume certain alignment for pointers
#pragma GCC ivdep	compile with -ftee-vectorize	tells compiler to vectorize a loop if loop not vectorized due to possible aliasing
#pragma omp simd	compile with -fopenmp-simd	openMP optimization supported outside within GCC annotations
#pragma GCC optimize x	x= unroll-loops, Ofast, O3, fast-math, inline	global optimization options
#pragma GCC target x	x= avx, avx2,fma,bmi,bmi2, lzcnt,popcnt,tune=native	global optimization options

TABLE 4. AMD HLS annotations explored in this work [2].

Type	Attribute	Additional options
Function Inlining	pragma HLS inline	off, on
Interface Synthesis	pragma HLS interface	mode= ap_fifo ,s_axilite, m_axi; port; bundle; offset; depth
Pipeline	pragma HLS Pipeline	rewind; enable_flush; II
Loop Unrolling	pragma HLS unroll	factor
Array Optimization	pragma HLS array_partition	variable; dim; factor type=cyclic,complete,block;

TABLE 5. Intel HLS annotations explored in this work [3].

Type	Attribute	Additional options
Pointer Interface	var_type*	restrict, volatile
Host Interfaces	ihc::mm_host< var >&var	configurations=Data Width, Stable=1,0, Alignment, Address Width, Burst Width, Latency
Agent Memories	agent registers, agent memories	configurations= hls_numbanks hls_writeonly, hls_singlepump, hls_doublepump, hls_bankwidth, volatile
Constant Interfaces	hls_stable_argument	hls_avalon_agent_register_argument
Loop Unrolling	#pragma unroll	factor
Max Concurrency	#pragma max_concurrency	1,0
Max Interleaving	#pragma max_interleaving	1,0
Other Loop Optimizations	#pragma ivdep #pragma loop_coalesce #pragma ii #pragma disable_loop_pipelining	factor
Global Optimizations	#pragma clang fp contract	fast,on,off
Other Global Optimizations	hls_disable_component_pipelining hls_max_concurrency hls_use_stall_enable_clusters	-

B. TARGET HARDWARE AND COMPILERS

1) CPU AS THE TARGET HARDWARE

The model evaluation and data collection are performed on a single multi-core CPU node. The CPU is a 16-core AMD Ryzen Threadripper PRO 5955WX, with 196GB of RAM. For CPU optimizations, we have used GCC-11.3.1-3. The codes are compiled using the `-O2 -std=c99 -ftee-vectorize -mcpu=zver3` flags; the assembly codes are then piped directly into `llvm-mca` [54] for performance analysis. Note this is just for proof-of-concept and user can use other performance analysis methods such as size of binary, number of instructions etc. Additional compilation flags that are required by certain annotations are encoded by the OSG during the Annotation Generation phase. For example, for GCC, `#pragma omp simd` allows users to take advantage of OpenMP's thread parallelization, especially for loops that do not auto-vectorize when the `-O3` flag is applied. This pragma requires an additional flag `-fopenmp-simd` in order to work. The OSG ensures that the right flags are supplied together with the annotation.

The compiler configurations explored in this work are given in Table 3. Aliasing analysis is an important part of an optimizing compiler since the C language, in general, does not prohibit overlapping memory regions. This information can be conveyed by inserting the pragma `__builtin_assume_aligned` at the appropriate location and

labelled with appropriate pointer type and variable names. Similarly `#pragma GCC ivdep` can be used by the programmer to assert that there are no loop carried dependencies and, hence, that the instructions can be executed in parallel. GCC supports some global annotations that can be inserted at the beginning of the code and applied to all the functions that follow. These include `#pragma GCC optimize(options)` and `#pragma GCC target(options)`. Another, `#pragma GCC optimize (Ofast)`, tells the compiler to assume associativity and hints about auto-vectorization. `#pragma GCC target` allows the compiler to make use of certain instruction set extensions. These can favor execution of other pragmas or generate SIMD instructions. However, while these annotations are used to increase application performance and exploit parallelization and vectorization, they can also cause a program to crash or make the code slower.

2) FPGA AS THE TARGET HARDWARE

For FPGA optimizations, AMD HLS (Xilinx Vitis Version 2022.1) and Intel HLS (Version 23.3) are used. The AMD target device is an `xc7a100tfgg676-2L`; the Intel target devices are Stratix 10 and Cyclone 10GX. The target frequency is 300 MHz for both tools. The annotations explored for AMD HLS are given in Table 4; those for Intel HLS in Table 5.

We note that the scope of this study does not include task level pipelining and structure packing using pragmas such as `pragma HLS dataflow` as discussed in Section IV-B. `#pragma HLS inline` allows the user to inline a function and its ports. However, inlining can also worsen performance, specifically when the inlined function needs to be called multiple times within the parent function [2]. Similarly, appropriate interfaces and their options must be defined for each function variable since interface contention will cause a bottleneck: e.g., for a RAM that allows fewer reads/writes in the same iteration. Also, `#pragma HLS unroll` allows users to unroll loops. Unrolling introduces hardware duplication. However, the amount of unrolling is constrained by the memory bandwidth. Hence this can speed up the computation, but can also cause deadlock.

For Intel HLS, one way to mitigate poor performance from the Avalon Memory Mapped Host Interfaces is to configure its properties such as data width, max burst, alignment, and read latency. If Avalon Memory Mapped Agent Memories are created for the specific variable, then their corresponding memory attributes—such as number of memory banks, bank width, readwrite_mode, and single versus double pump—can be optimized. Similarly, the HLS compiler allows the

TABLE 6. RL setup and PPO hyper-parameters.

Component	Details
Software and Environment Setup	
RL Environment Interface	OpenAI Gym 0.21.0
RL Agent Framework	Ray 1.7.0 + RLlib
ML Backend	TensorFlow 2.6.0 + Keras 2.6.0
Programming Language	Python
Hardware	Single-worker
Training Iterations	300
Random Seed	3
PPO Hyperparameters (fixed across experiments)	
Learning Rate (α)	5e-5
Discount Factor (γ)	0.99
GAE Lambda (λ)	0.95
Clip Parameter	0.3
Value Function Clip	10.0
Entropy Coefficient	0.0
SGD Minibatch Size	128
Training Batch Size	4000
Optimization Epochs	30
Rollout Fragment Length	200
Number of Workers	single worker
Policy Hidden Layers	[256, 256]
Deep Learning Framework	TensorFlow (tf)

Algorithm 2 RL-Based Annotation Optimization

Given: Source code S , Compiler Configuration C for Target Architecture A
Output: Optimized Annotated Code S_{opt}
Generate unique code annotations F from S and C
Initialize RL environment with F , S and C
Initialize PPO agent with default hyper-parameters
for each episode **do**
 Append the annotation decision using policy π_θ , every step of the given episode, into an annotation set A_t
 Apply A_t to S to obtain annotated code S_t
 Compile S_t using C for target A
 Measure performance and compute reward R_t
 Update policy π_θ using reward R_t
end for
Select A_t with best observed reward after the annotated code passes the validation step
Return annotated code S_{opt} using A_t

use of the *volatile* keyword to allow concurrent agent memory accesses and the *restrict* keyword to prevent memory dependencies. Loops can be coalesced to reduce area and latency. The `#pragma disable_loop_pipelining` allows users to avoid loop-carried dependencies by generating sequential datapaths.

In Section VII-C, we test the approach on common FPGA workloads that have been used in previous work [10] and demonstrate how code annotation compares with manual source code transformations. To obtain the baseline, the applications are all transformed into CPU-like single kernel C code, the standard starting best practice (Version 1 in Table 2). Because of the often orthogonal capabilities

TABLE 7. Bayesian optimization setup and hyper-parameters.

Component	Details
Optimization Framework	BoTorch 0.8.0 + Ax 0.2.10
Surrogate Model	Gaussian Process Regression
Acquisition Function	Expected Improvement (default)
Initial Random Samples	10 (default)
Optimization Iterations	1000
Gradient Usage	Gradient-Free for target function
Random Seed	Fixed (constant across runs)

of programmer optimization versus code annotations (as described in Section IV) we also apply AnnotationGym to codes that were transformed using the methods in Table 2.

C. EVALUATED OPTIMIZERS

AnnotationGym evaluates using two different optimizers.

1) REINFORCEMENT LEARNING (RL)

As discussed in Section III, RL is an ML method where an agent learns by interacting with its environment. This work utilizes PPO [55], an on-policy, model-free RL technique with the goal of learning a policy, i.e., a state-to-action mapping that maximizes rewards accumulated over time. We propose that an RL agent can capture code characteristics and compiler annotations that work well together and so enable the learning of a model that predicts best code insertions for a given application.

RL is set up using Open AI Gym (0.21.0) to provide the unified environment interface, Ray (1.7.0) to provide the unified API for RL, and its RLlib library to provide the agent interface. Tensorflow (2.6.0) is used for ML tasks, together with Keras (2.6.0) to provide the neural net API for Python (3.9). To ensure standardization, each test is run using a *single worker* on a standard multi-core CPU, a total training time of *300 iterations*, and the same initial *seed* value for random number generation. Table 6 summarizes the RL environment and the hyper-parameters used for PPO. All values are kept the same for all experiments to ensure reproducibility and clarity. However, the user can adjust these according to their application, e.g., using Ray Tune for hyper-parameter tuning. Details for the RL Optimizer are as follows.

Action: Each action represents a single annotation decision. This could be either one of the annotations generated by the pragma generator or a null action specifying that the agent do nothing. These decisions are appended to an ordered list of actions for the episode. The list of actions suggested by the PPO agent is passed to the OSG after every episode. This inserts the actions into appropriate locations in the source code and outputs an annotated code.

State: A histogram of applied annotations is used to represent the state; this is referred to as an *action histogram*. To implement this histogram, a list is initialized of length equal to that of the optimization space generated by the OSG; each element of the list is incremented and updated when a pragma corresponding to the pragma number is applied.

Algorithm 3 BO-Based Annotation Optimization

Given: Source code S , Compiler C of the Target Architecture A
Output: Optimized Annotated Code S_{opt}
Generate unique code annotations F from S and C
Define search space $\mathcal{D}$ from F
Initialize BO framework with surrogate model
Collect initial random samples and evaluate performance
for each optimization iteration **do**
 Fit surrogate model to collected data
 Optimize acquisition function, output annotation set A_t
 Apply A_t to S to obtain annotated code S_t
 Compile S_t using C for target A
 Measure performance and update dataset
end for
Select A_t with best observed performance after the annotated code passes validation
Return annotated code S_{opt} using A_t

The agent learns to map the distribution of the number of pragmas applied to the next pragma that should be applied. The learning can be seen from the overall increase in the average sum of rewards across episodes. Since a possible action can also be a null pragma, the agent is simultaneously forced to not only learn the optimal combination, but also an optimal number of pragmas to insert.

Reward is defined as the difference in latency between the unoptimized code and the annotated code in the current step; lower latency results in a higher reward value. The reward is set to a default value of 0 for each step, except for the last step in the episode in which the actual latency is obtained from the HLS tool. *Maximum Reward* is defined as the highest reward obtained by any episode during training.

Episode size: The episode size is fixed to the number of possible pragmas for the code, i.e., the length of the action space. This also bounds the size of the action histogram.

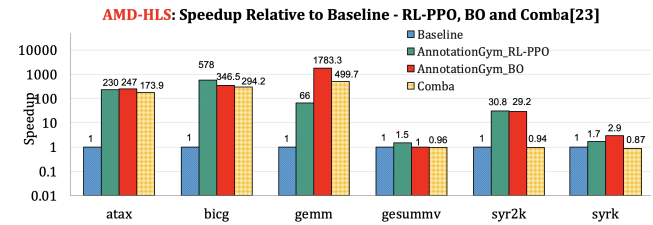
Algorithm 2 gives pseudo code for the RL-based optimization process. The agent iteratively explores the annotation space to maximize performance rewards.

2) BAYESIAN OPTIMIZATION (BO)

BO is a framework for sample-efficient gradient-free optimization of functions. At a high level, the framework operates by alternating between using a surrogate model of the function to pick a new point to sample and updating the surrogate model. The surrogate model is usually a Gaussian process. The search space consists of all possible combinations of annotations with their associated factors and options inserted into the code. In AnnotationGym, BO is setup using BoTorch 0.8.0 and ax-platform 0.2.10. Table 7 summarizes the BO setup and other configurations used in our experiments. Note that unlike PPO, BO does not have fixed hyper-parameters. Instead it has common

TABLE 8. Speedups factors achieved for GCC for two optimizers.

Application	RL-PPO	RL-BO
2mm	1.41	1.41
3mm	1.07	1.07
adi	1.26	1.26
atax	1.80	1.80
bicg	1.00	1.00
correlation	1.06	1.06
covariance	1.45	1.50
deriche	1.42	1.42
doitgen	1.33	1.33
durbin	1.21	1.00
fdtd-2d	2.57	2.45
floyd-warshall	2.39	2.39
gemm	1.08	1.08
gemver	1.56	1.56
gesummv	1.19	1.19
gramschmidt	1.43	1.43
heat-3d	2.75	2.75
jacobi-1d	5.27	5.27
jacobi-2d	2.28	2.28
syrk	1.49	1.49

**FIGURE 6.** Speedup achieved by using AnnotationGym for AMD HLS with 2 different Optimizers, RL-PPO and BO, and comparison to previous work. Baseline is the unoptimized code [29].

configurations that the user can vary. In this work, we used the default setting for all experiments.

Algorithm 3 describes the Bayesian Optimization-based approach, where a surrogate model guides the search for optimal annotations.

VII. RESULTS

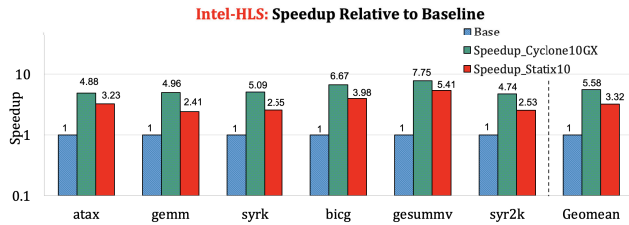
In this section we present the results and our findings. Section VII-A, gives the results obtained by optimizing common CPU benchmarks using AnnotationGym. The annotations explored in this work are given in Table 3. The benefits are obtained largely from enabling use of simd instructions; easier loop optimizations such as unrolling, tiling and prefetching; removing conservative compiler checks; improving memory usage by eliminating misaligned memory stalls and removing the need for runtime alignment checks. Section VII-B discusses results obtained when AnnotationGym is applied on common benchmarks. Section VII-C discusses when AnnotationGym is applied on codes hand optimized using standard best practices for FPGAs.

A. CPU

Table 8 shows speedups obtained by annotating benchmarks from Polybench/C using AnnotationGym. The results are

TABLE 9. Post-place-and-route results (xc7a100tfgg676-2L).

Application	Runtime (μ s)	Freq (MHz)	Latency (cycles)	LUTs	BRAMs	FF	DSPs
atax_Unoptimized	251	173	43472	35660	31	81294	96
atax_Optimized	0.9	215	190	6805	47	15850	96
bicg_Unoptimized	-	-	163564	-	-	-	-
bicg_Optimized	2.3	174	397	13045	92	29194	192
gemm_Unoptimized	5223	130	677635	35846	31	82431	51
gemm_Optimized	208.6	214	44657	16037	31	44792	51
gesummv_Unoptimized	1.1	239	263	4884	31	11971	102
gesummv_Optimized	0.9	195	174	9898	77	21766	102
syr2k_Unoptimized	3374	213	720433	8878	31	24435	9
syr2k_Optimized	297	233	69296	5581	32	16554	9
syrk_Unoptimized	3567	202	720481	7989	31	21820	6
syrk_Optimized	2999	230	689696	6538	17	13598	24

**FIGURE 7. Speedups using AnnotationGym for Intel HLS. Base is the unoptimized code.**

evaluated by running 100 iterations of each kernel using the tool `llvm-mca` [54] and comparing the latency. The base codes are the unoptimized source code. The speedups show how much gain is possible using the different Optimizers within AnnotationGym. We observe a $1.8\times$ average increase in performance when using simple annotations listed in Table 3.

For most benchmarks RL-PPO and BO give the same speedup. However, for `fdtd-2d` and `durbin`, RL-PPO outperforms BO. BO allows users to set optimizer-specific configurations such as *bias*. Similarly, RL-PPO allows users to configure hyperparameters. Tuning these Optimizer-specific configurations per benchmark can improve the learning rate.

AnnotationGym annotates the code with the discovered annotations and ensures through robust validation that the annotations preserve codes' semantics and function. Our results have shown, while an intelligible annotation can enhance performance by a geomean of $1.8\times$, a bad annotation set can degrade the performance by an average of $2.8\times$ (up to $5.48\times$) across the evaluated benchmarks [11].

B. FPGA ON UNOPTIMIZED CODE

In the current study we have used both AMD HLS and Intel HLS. The annotations included in the compiler configuration for each HLS compiler are given in Tables 4 and 5. After every episode the annotated code is fed into the HLS tool and the post-synthesis reports generated are parsed by the toolflow to output the latency values. In this section, we evaluate how AnnotationGym can be used to optimize codes from common benchmarks, that are currently in their unoptimized(unchanged) form. In the next section, we evaluate how common benchmark codes, that are hand

TABLE 10. Place-and-Route Results (Cyclone 10 GX).

Application	Runtime (μ s)	Freq (MHz)	LUTs	Dedicated Registers	Block Mem Bits	DSP Blocks
atax_Unoptimized	0.9	266	5755	10765	34624	5
atax_Optimized	0.1	388	1423	3257	8192	4
bicg_Unoptimized	0.8	273	5407	9779	45248	5
bicg_Optimized	0.08	414	1812	3467	33792	4
gemm_Unoptimized	0.7	351	4502	8865	34176	9
gemm_Optimized	0.2	234	1162	2505	24576	6
gesummv_Unoptimized	0.95	293	6563	11998	70272	8
gesummv_Optimized	0.09	397	1789	3363	16384	7
syr2k_Unoptimized	0.7	323	6321	12150	36864	11
syr2k_Optimized	0.2	231	1942	4908	24576	8
syrk_Unoptimized	0.7	333	5182	9858	32768	8
syrk_Optimized	0.2	235	1255	2861	16384	6

optimized using standard best practices, can also benefit from AnnotationGym.

1) SPEEDUP - AMD

AnnotationGym explores design spaces that are exponentially larger (on average $1.34E+8\times$) than in the previous work [29]. This is because the annotations considered include both typical pragmas, such as loop unroll and array partitioning, and also interface-related pragmas and their parameters (offset, bundle, port, depth). These parameters specify how RTL ports are created from the function's arguments. Similarly, pipelining can also have multiple options. The pragmas are elaborated in Table 4. RL allows exploration of a much larger design space efficiently. This is because RL agent is continuously learning from its experience and hence can intelligently scan the design space.

Fig. 6 shows the speedup of different Optimizers within AnnotationGym and a comparison to previous work Comba [29] that (1) used the same benchmark [9] to quote their performance advantage (2) we were able to port their work and evaluate it for comparison values. *Base* is defined by the unoptimized benchmark. The speedup is measured with respect to the base. For some applications—such as `bicg`, `gesummv`, and `syr2k`—the Optimizer RL-PPO within AnnotationGym gives the best results. For others—`atax`, `gemm` and `syrk`—bayesian optimization (BO) gives the best results. AnnotationGym speeds up all applications by a geomean average of a $56.9\times$. This is a factor of $2.72\times$ performance improvement over previous work.

Place-and-Route: Table 9 gives the results when each unoptimized and AnnotationGym-optimized-codes is implemented on the target device. We make several observations. (i) The runtimes of the actual circuits improve on average by $64\times$, hence validating the performance goal. (ii) Generally the resource utilization improves and frequency of the optimized circuits after place-and-route is better than the unoptimized circuits. (iii) For `gesummv`, the frequency of the optimized circuit decreases and resources such as number of LUTs, BRAMs, and FFs increase. Yet the runtime is reduced by $1.24\times$ and latency by $1.5\times$. This shows that the overall

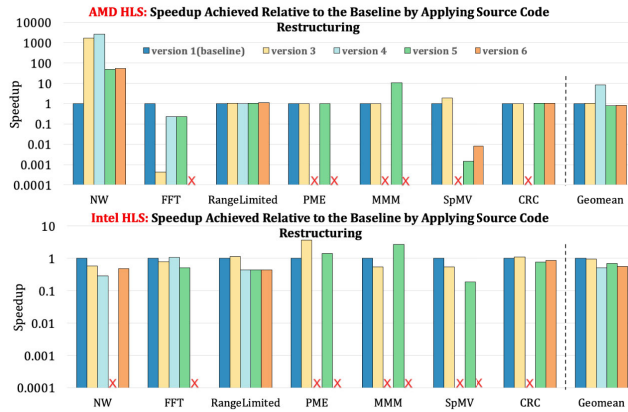


FIGURE 8. Systematic optimizations performed in [10] are applied. 'X' indicates that either the version was not created since the corresponding optimizations did not exist for the specific benchmark or the tool run into a deadlock/compilation error.

goal of achieving speedup has been achieved in the way circuits have been optimized. (iv) For bigc, the unoptimized code fails to generate hardware in the export-ip phase. This is in spite of it being able to pass co-simulation run in AMD HLS. This further highlights the importance of annotations for HLS: it is sometimes impossible to generate circuits without *legal* pragmas.

2) SPEEDUP - INTEL

Fig. 7 shows the speedups achieved by annotating the various codes for Intel HLS using AnnotationGym. Results are obtained for both Stratix 10 and Cyclone10GX. For the Stratix 10 performance is improved by a geomean of $3.21\times$; for the Cyclone 10GX by $5.6\times$. An important observation is that the annotations are different across applications. For example, for atax, optimal performance is achieved by declaring some pointer variables as host memory and others as agent memory. For bigc, performance is enhanced by leveraging the volatile keyword and memory coalescing on agent memory. For gemm and syrk, loop coalescing and instantiation of Avalon register arguments enhances performance. Since, to the best of our knowledge(at the time of this submission), this is the only work exploring code optimization using pragmas for Intel HLS no comparison is possible.

Place-and-Route: All optimized results are verified by generating bitstreams and running the design on the actual hardware (for Cyclone 10 GX). Results are given in Table 10. We observe the following. (i) In all cases, AnnotationGym achieves its performance target of a decrease in latency (in cycles) as shown in Fig. 7. (ii) The overall runtime (in μs) decreases for all cases. Frequency, however, does decrease in some cases due to longer critical paths. However, since the reduction in latency is significantly larger than the decrease in frequency, the overall runtime decreases significantly. (iii) Optimization drastically improves resource utilization. As a

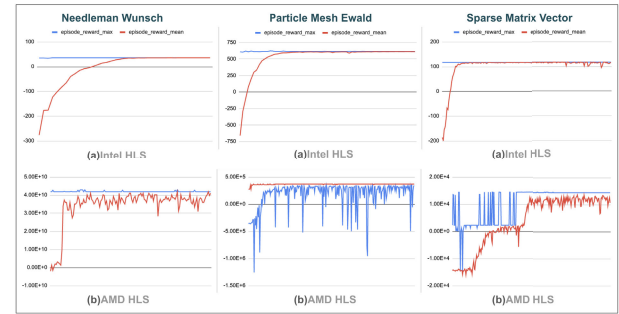


FIGURE 9. RL Episode Reward Max and Episode Reward Mean. The rising mean shows how the RL agent learns to annotate code that increase its overall reward (decrease in latency).

result, we can target larger problem sizes than unoptimized codes and fit more logic into the same device.

C. FPGA ON OPTIMIZED CODE

In this section we evaluate AnnotationGym on common FPGA workloads that have been used to highlight advantage of manual source code reconstruction in [10]. The approach to hand optimizing codes has been discussed in Section IV-B. As discussed in Section VI-B2, we demonstrate how code annotation compares with manual source code transformations, and show how further improvements over best hand optimized codes is possible. Some details are as follows. First, Version 2 from Table 2 is omitted. It involves leveraging task-level parallelism using multiple kernels; the focus here is on single kernel optimization. Moreover, our premise is that the HLS compiler is still able to automatically infer task parallel pipelines [10]; also, having a single kernel offers several advantages such as reduced computation overhead and simple algorithmic flow [56].

Impact of HLS Compiler: Figure 8 shows the results of individual optimizations proposed by [10] performed on the baseline code (Version 1) as evaluated using the AMD HLS and Intel HLS tools [12]. In [10] the authors observed that each subsequent version of the source code restructuring typically increases performance. We find here that source code rewriting does not give much benefit at all for multiple benchmarks including FFT, RangeLimited, and CRC. For AMD HLS, the benefit is seen largely in the systolic-array-based Needleman-Wunsch benchmark, where each version gives better performance than the baseline. Even here, however, Versions 4 and 5 unexpectedly give poor results when compared to prior work. For Intel HLS, in the case of PME, Version 3 optimizations give the best results. For MMM, for both Intel and AMD, Version 5 gives best performance.

These findings highlight a crucial observation: **the benefits of programmer-applied optimizations, e.g., based on best practices, are highly compiler dependent.** For example, pipeline stages are inferred differently by both HLS tools. Performing computations in detail by storing intermediate results in pipeline registers has little to no

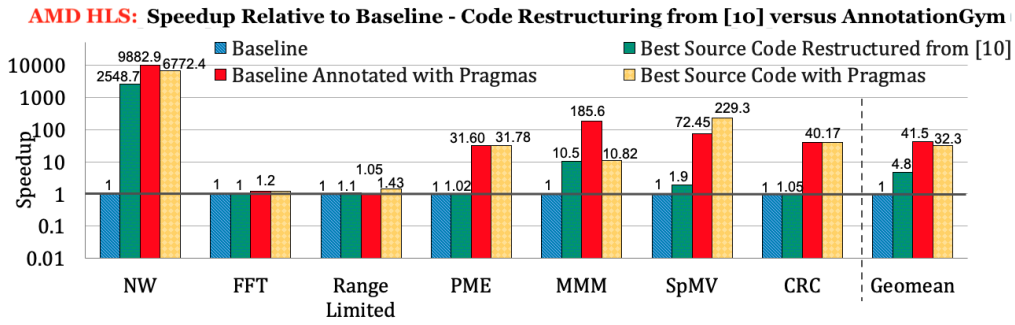


FIGURE 10. The performance of proposed annotation framework, AnnotationGym on baseline source codes compared to source code restructuring proposed in [10].

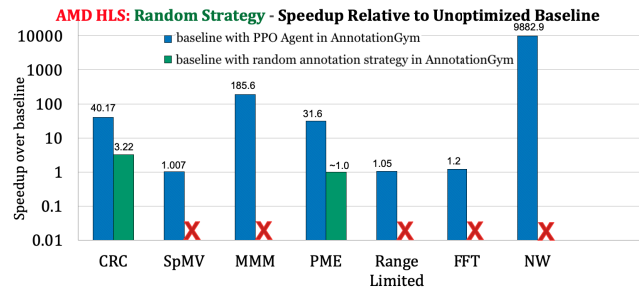


FIGURE 11. Relative Speedup achieved by using base configuration (with PPO agent) in AnnotationGym versus using a random strategy to annotate code for equivalent number of iterations. 'X' is used to denote the case when not even a single episode of random annotations output compilable code; the proposed annotations were outputting compilable errors on AMD HLS.

AMD HLS: Speedup Achieved by Applying AnnotationGym to Restructured Source Code for each Benchmark (Speedup is Relative to Unoptimized Code Structure)

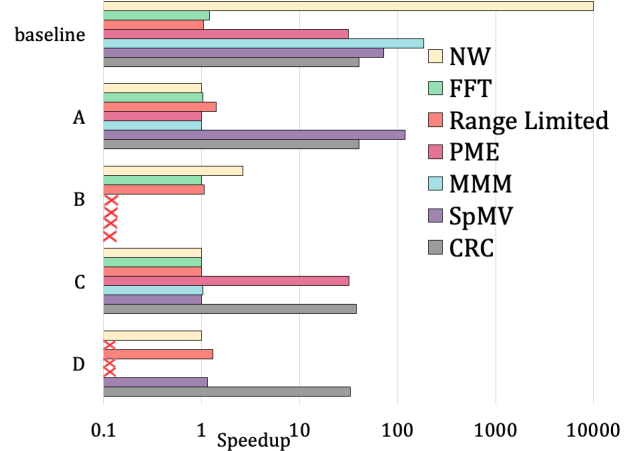


FIGURE 12. Impact of applying AnnotationGym to each starting code version. 'X' indicates that code structure was not created since the corresponding optimizations did not exist for the specific benchmark. The speedup >1 shows performance was enhanced by applying strategy of AnnotationGym. Even for the worst case, performance is equal to the unoptimized code. Baseline is the same as Version 1. Code structure A refers to version 3, code structure B refers to version 4, code structure C refers to version 5, code structure D refers to version 6 from Table 2.

impact for AMD HLS, but results in improvements in some cases for Intel HLS. This result is also heavily dependent on the application: dissolving branch statements into conditional assignments gives performance improvement for MMM; for other applications, however, there is little benefit. Below we discuss results specific to AMD and Intel compilers.

Episode Reward Mean for RL Agent Training: Figure 9 highlights how the PPO agent learning is improving over time as the model learns to optimize the given code. Initially, the episode reward mean is mostly negative, showing how most of the annotation applied on the code lead to performance degradation. **Over time, the agent learns which annotations work best for a certain code and the episode reward mean converges close to the best possible episode reward seen for a given code**(the max episode reward is given in blue and the episode reward mean in red are converging towards it).

1) SPEEDUP - AMD

Figure 10 demonstrates the effectiveness of AnnotationGym when compared to source code restructuring proposed in [10] for AMD HLS. We postulate that modern HLS compilers can do a decent job at optimizing C codes without additional source code restructuring and using HLS tool specific annotations only. The results in Figure 10 support this claim for AMD HLS. Two versions are compared with the baseline.

In the first, the baseline code for each of the seven FPGA benchmarks is optimized using source code rewriting; the code version that gives the maximum speedup is selected. In the second, AnnotationGym is applied to the baseline code. The results show that the RL agent does a good job at annotating code to enhance performance. On average, AnnotationGym improves the baseline codes by $42\times$ versus source code restructuring that improves it by $4.8\times$. At worst, AnnotationGym gives performance comparable to source code restructuring. This also answers one of the big questions in HLS: Is it possible to perform better than manual source code restructuring using compiler annotations? The answer, as backed by the results is Yes. For some applications such as NW, PME, MMM, SpMV and CRC this is by a large margin. For the balance, FFT and Range Limited, it is still possible to give slightly better (by 20%) or, at least, comparable performance (within 5%).

Do random annotations also improve performance? As discussed in Section V, AnnotationGym uses Proximal Policy

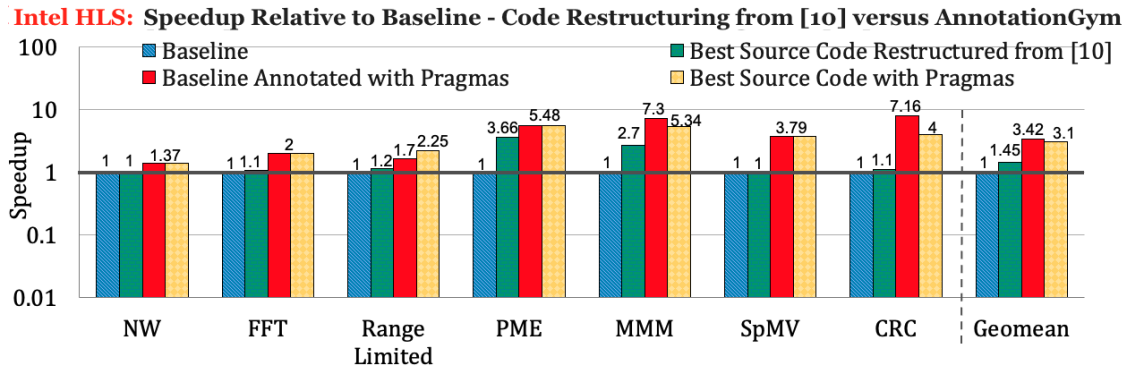


FIGURE 13. Speedup achieved by using AnnotationGym to insert Intel HLS pragmas for both baseline and best source code restructured versions.

Optimization (PPO) as the RL agent. PPO updates its existing policy at each *step* to minimize the cost function without deviating too much from the previous policy. The goal here is to understand how much of the performance gain (in Figure 10) is because of RL (PPO in this case); to do this we compare with a baseline *random* strategy. For the random strategy, we replaced the PPO agent with simple nested loops that ran for an equivalent number of steps, episodes, and iterations. In each step of the PPO-RL workflow, the agent selects one pragma. After completing an episode of ‘n’ steps, a total of ‘n’ pragmas have been applied, and their combined effect is captured as a reward. In the random baseline, the agent instead selects ‘n’ pragmas at random in each episode. This process is repeated ‘m’ times, where ‘m’ is the number of episodes in one iteration. For each iteration, the average reward across episodes is computed and recorded. The procedure is then repeated over multiple iterations, with the mean reward per iteration tracked.

The mean reward per iteration is then compared with the case when PPO policy is used instead. The results are given in Figure 11. For five out of seven benchmarks, the random strategy could not get even a single random occurrence of annotations without compilation errors. This is because for AMD HLS, only a small subset of pragmas/annotations can work well together. And since the random strategy does not learn over time, for every iteration it ran into errors. For CRC and PME it was possible to get legal annotations even with random strategy, perhaps because the action space of effective annotations is so small that it is possible to pick the acceptable ones just by chance. In both cases, however, the resulting performance was indistinguishable from the original baseline code.

Combining code restructuring with AnnotationGym Figure 12 shows the results of combining source code rewriting [10] with AnnotationGym. For the established FPGA benchmarks, we choose as our starting points different code versions as proposed in Table 2. Code structures A-D refer to versions 3-6. Our goal is to understand how much

performance potential is achievable when AnnotationGym is applied to each code structure. The results show:

- Figure 8 gives the best source code version for each application, e.g., for NW, on AMD tools, it is version 4. Applying annotations on top of version 4 (code structure B) gives a speedup of $2.67\times$ as can be seen in Figure 12. For SpMV, we can improve the best-case hand optimized code (code structure A-version 2 from Figure 8) using AnnotationGym by $120\times$. On average, AnnotationGym improves the best hand optimized source codes by around $32.3\times$ with respect to the unoptimized baseline.
- Results in Figure 12 show that pre-processing using AnnotationGym gives the maximum performance benefit when applied to baseline code without any source code restructuring. Exceptions include Range Limited where AnnotationGym applied to code structures A and D gives slightly better performance than when it is applied to baseline codes; and SpMV where AnnotationGym applied to code structure A gives $3.16\times$ better performance than when it is applied to the baseline code. AnnotationGym improves baseline codes by $42\times$, code structure A by $3.54\times$, code structure B by $1.42\times$, code structure C by $2.77\times$ and code structure D by $2.65\times$.
- From the results we can also conclude that the HLS compiler in general, does well at inferring pipeline stages and leveraging parallelism; further benefit can be achieved by applying tool-specific annotations only (something that AnnotationGym does) without requiring to manually rewrite the code structure. This is especially evident in the case of CRC. AnnotationGym applied to every code structure results in nearly identical performance improvement. It is also evident in PME where AnnotationGym applied to the baseline and to code structure C give the same performance improvement.

2) SPEEDUP - INTEL

Combining code restructuring with AnnotationGym For Intel HLS, hand optimized versions as proposed in Table 2 were first run on the tool. The results are given in Figure 8. Out of these, the maximum performance improvement possible using hand optimizations (source code restructuring) for each workload is selected and its performance advantage is displayed as green bar in Figure 13 [12]. Next, the baseline codes for each workload are annotated using AnnotationGym and the performance improvement is given by the red bars. Lastly, the best source code restructured codes are run through AnnotationGym and the performance improvement possible is given by the yellow bars. We note that in most cases, annotating the baseline codes gives performance improvement either better than, or at least comparable to annotating the best source code restructured codes. AnnotationGym improves the performance of baseline codes by a geomean of $3.42\times$ and best source code restructured by $3.1\times$.

D. DISCUSSION

Overall, the results are promising: benefits are found for unoptimized codes; these persist for hand-optimized best codes. As expected, annotations guide various HLS compiler decisions and add substantially to the code optimization process. More specifically, we see that the performance benefit comes in large part from many sources: (i) Loop Unrolling and Pipelining; (ii) Memory Management (i.e. defining data locality, memory access patterns, data reuse); (iii) Specifying Interfaces to increase throughput (i.e. communication width, communication protocols); (iv) Resource Binding (specifying FPGA memory such as DSP, BRAM, and External memory to be used for specific parts of the code); (v) Parallelism (parallel regions within the code) and dataflow optimization. But the overall benefit is apparently derived from the creation of a large search space and its effective traversal.

AnnotationGym makes a unique scientific contribution by demonstrating how ML and learning based algorithms can be used to automate the insertion of annotations both for CPU and FPGA codes. It shows the breadth of the approach with micro-benchmarks from Polybench and common FPGA workloads. Future work could explore how AnnotationGym performs as the size and complexity of the codebase increases. Since it was meant to demonstrate the viability of an approach, further work can address how it would scale for larger projects with multiple complex data structures.

We have noticed AnnotationGym may take a long time to converge to a good solution, particularly for complex codes such as for the FFT. This is largely because we use the underlying HLS tool to compute the reward. If a developer requires faster turnaround time, they can replace the underlying tool with a Quality-of-Results estimator or a neural-net-based cost model. This may sacrifice accuracy but guarantee a much faster convergence of results.

The results in this work highlight the significant variability in the performance improvements observed across compilers, architectures, and even applications and annotations.

VIII. CONCLUSION

We have developed an automatic source code annotation framework that replaces developer expertise and effort with RL. Perhaps the most important broader contribution of AnnotationGym is its generality and extensibility, allowing continued research across compilers. In particular, it provides a platform to test, train, and evaluate different compilers and target devices under one framework using limited domain expertise for setup and usage. AnnotationGym will be open-source and publicly available and we will work with alpha/beta users on the integration process.

REFERENCES

- [1] CodeForces. (2023). *Gcc Optimization Pragmas*. Accessed: Dec. 21, 2024. [Online]. Available: <https://codeforces.com/blog/entry/96344>
- [2] AMD Xilinx. (2022). *Vitis High Level Synthesis User Guide*. Accessed: Dec. 10, 2024. [Online]. Available: <https://www.xilinx.com/support/documents/swmanuals/xilinx2022/ug1399-vitis-hls.pdf>
- [3] Intel. (2023). *Intel High Level Synthesis Compiler*. Accessed: Oct. 2024. [Online]. Available: <https://www.intel.com/content/www/us/en/software/programmable/quartus-prime/hls-compiler.html>
- [4] X. Wu, M. Kruse, P. Balaprakash, H. Finkel, P. Hovland, V. Taylor, and M. Hall, "Autotuning PolyBench benchmarks with LLVM clang/polly loop optimization pragmas using Bayesian optimization," *Concurrency Comput., Pract. Exper.*, vol. 34, no. 20, p. 6683, Sep. 2022.
- [5] C. Cummins, B. Wasti, J. Guo, B. Cui, J. Ansel, S. Gomez, S. Jain, J. Liu, O. Teytaud, B. Steiner, Y. Tian, and H. Leather, "CompilerGym: Robust, performant compiler optimization environments for AI research," in *Proc. IEEE/ACM Int. Symp. Code Gener. Optim. (CGO)*, Apr. 2022, pp. 92–105.
- [6] A. Haj-Ali, N. K. Ahmed, T. Willke, Y. S. Shao, K. Asanovic, and I. Stoica, "NeuroVectorizer: End-to-end vectorization with deep reinforcement learning," in *Proc. 18th ACM/IEEE Int. Symp. Code Gener. Optim.*, Feb. 2020, pp. 242–255.
- [7] M. Trofin, Y. Qian, E. Brevdo, Z. Lin, K. Choromanski, and D. Li, "MLGO: A machine learning guided compiler optimizations framework," 2021, *arXiv:2101.04808*.
- [8] S. Che, M. Boyer, J. Meng, D. Tarjan, J. W. Sheaffer, S.-H. Lee, and K. Skadron, "Rodinia: A benchmark suite for heterogeneous computing," in *Proc. IEEE Int. Symp. Workload Characterization (IISWC)*, Oct. 2009, pp. 44–54.
- [9] L.-N. Pouchet. (2012). *Polybench: The Polyhedral Benchmark Suite*. [Online]. Available: <http://www.cs.ucla.edu/pouchet/software/polybench>
- [10] A. Sanaullah, R. Patel, and M. Herbordt, "An empirically guided optimization framework for FPGA OpenCL," in *Proc. Int. Conf. Field-Programmable Technol. (FPT)*, Dec. 2018, pp. 46–53.
- [11] H. Shahzad, "Optimizing the optimizer increasing performance efficiency of modern compilers," Ph.D. dissertation, Dept. Elect. Comput. Eng. ENG, Boston Univ., Boston, MA, USA, 2025.
- [12] H. Shahzad, A. Sanaullah, S. Arora, U. Drepper, and M. Herbordt, "AutoAnnotate: Reinforcement learning based code annotation for high level synthesis," in *Proc. 25th Int. Symp. Quality Electron. Design (ISQED)*, Apr. 2024, pp. 1–9.
- [13] A. H. Ashouri, G. Palermo, J. Cavazos, and C. Silvano, "The phase-ordering problem: A complete sequence prediction approach," in *Automatic Tuning of Compilers Using Machine Learning*. Cham, Switzerland: Springer, 2018, pp. 85–113.
- [14] H. Leather and C. Cummins, "Machine learning in compilers: Past, present and future," in *Proc. Forum Specification Design Lang. (FDL)*, Sep. 2020, pp. 1–8.

- [15] Z. Wang and M. O'Boyle, "Machine learning in compiler optimization," *Proc. IEEE*, vol. 106, no. 11, pp. 1879–1901, Nov. 2018.
- [16] H. Arabnejad, J. Bispo, J. M. P. Cardoso, and J. G. Barbosa, "Source-to-source compilation targeting OpenMP-based automatic parallelization of C applications," *J. Supercomput.*, vol. 76, no. 9, pp. 6753–6785, Sep. 2020.
- [17] J. de Fine Licht, M. Besta, S. Meierhans, and T. Hoefler, "Transformations of high-level synthesis codes for high-performance computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 5, pp. 1014–1029, May 2021.
- [18] H. R. Zohouri, N. Maruyama, A. Smith, M. Matsuda, and S. Matsuoka, "Evaluating and optimizing OpenCL kernels for high performance computing with FPGAs," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, Nov. 2016, pp. 409–420.
- [19] C. Dave, H. Bae, S.-J. Min, S. Lee, R. Eigenmann, and S. Midkiff, "Cetus: A source-to-source compiler infrastructure for multicores," *Computer*, vol. 42, no. 12, pp. 36–42, Dec. 2009.
- [20] G. Mendonça, B. Guimarães, P. Alves, M. Pereira, G. Araújo, and F. M. Q. Pereira, "DawnCC: Automatic annotation for data parallelism and offloading," *ACM Trans. Archit. Code Optim.*, vol. 14, no. 2, pp. 1–25, Jun. 2017.
- [21] N. Wu, Y. Xie, and C. Hao, "IronMan-pro: Multiobjective design space exploration in HLS via reinforcement learning and graph neural network-based modeling," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 3, pp. 900–913, Mar. 2023.
- [22] V. Sreenivasan, R. Javali, M. Hall, P. Balaprakash, T. Scogland, and B. R. d. Supinski, "A framework for enabling OpenMP autotuning," in *Proc. Int. Workshop OpenMP*, Sep. 2019, pp. 50–60.
- [23] C. Aubert, T. Rubiano, N. Rusch, and T. Seiller, "A novel loop fission technique inspired by implicit computational complexity," 2022, *arXiv:2206.08760*.
- [24] P. Ramos, G. Souza, D. Soares, G. Araújo, and F. M. Q. Pereira, "Automatic annotation of tasks in structured code," in *Proc. 27th Int. Conf. Parallel Archit. Compilation Techn.*, Nov. 2018, pp. 1–13.
- [25] M. Kruse and H. Finkel, "User-directed loop-transformations in clang," in *Proc. IEEE/ACM 5th Workshop LLVM Compiler Infrastruct. HPC (LLVM-HPC)*, Nov. 2018, pp. 49–58.
- [26] J. Koo, P. Balaprakash, M. Kruse, X. Wu, P. Hovland, and M. Hall, "Customized Monte Carlo tree search for LLVM/Polly's composable loop optimization transformations," in *Proc. Int. Workshop Perform. Modeling, Benchmarking Simulation High Perform. Comput. Syst. (PMBS)*, Nov. 2021, pp. 82–93.
- [27] GCC GNU. *Options That Control Optimizations*. Accessed: Jan. 14, 2025. [Online]. Available: <https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html>
- [28] A. Sohrabizadeh, C. H. Yu, M. Gao, and J. Cong, "AutoDSE: Enabling software programmers to design efficient FPGA accelerators," *ACM Trans. Design Autom. Electron. Syst.*, vol. 27, no. 4, pp. 1–27, Jul. 2022.
- [29] J. Zhao, L. Feng, S. Sinha, W. Zhang, Y. Liang, and B. He, "COMBA: A comprehensive model-based analysis framework for high level synthesis of real applications," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, Nov. 2017, pp. 430–437.
- [30] L. Ferretti, G. Ansaloni, and L. Pozzi, "Cluster-based heuristic for high level synthesis design space exploration," *IEEE Trans. Emerg. Topics Comput.*, vol. 9, no. 1, pp. 35–43, Jan. 2021.
- [31] Q. Sun, T. Chen, S. Liu, J. Chen, H. Yu, and B. Yu, "Correlated multi-objective multi-fidelity optimization for HLS directives design," *ACM Trans. Design Autom. Electron. Syst.*, vol. 27, no. 4, pp. 1–27, Jul. 2022.
- [32] M. Yu, S. Huang, and D. Chen, "Chimera: A hybrid machine learning-driven multi-objective design space exploration tool for FPGA high-level synthesis," in *Proc. Int. Conf. Intell. Data Eng. Automated Learn.*, Nov. 2021, pp. 524–536.
- [33] K. Youssef, N. Shah, M. Gokhale, R. Pearce, and W.-c. Feng, "AutoPager: Auto-tuning memory-mapped I/O parameters in userspace," in *Proc. IEEE High Perform. Extreme Comput. Conf. (HPEC)*, Sep. 2022, pp. 1–7.
- [34] OpenAI. (2017). *Proximal Policy Optimization*. Accessed: Nov 7, 2024. [Online]. Available: <https://openai.com/research/openai-baselines-ppo>
- [35] P. Amiri, A. Pérard-Gayot, R. Membarth, P. Slusallek, R. Leiße, and S. Hack, "FLOWER: A comprehensive dataflow compiler for high-level synthesis," in *Proc. Int. Conf. Field-Programmable Technol. (ICFPT)*, Dec. 2021, pp. 1–9.
- [36] J.-M. Gorius, S. Rokicki, and S. Derrien, "SpecHLS: Speculative accelerator design using high-level synthesis," *IEEE Micro*, vol. 42, no. 5, pp. 99–107, Sep. 2022.
- [37] A. Sohrabizadeh, Y. Bai, Y. Sun, and J. Cong, "Automated accelerator optimization aided by graph neural networks," in *Proc. 59th ACM/IEEE Design Autom. Conf.*, Jul. 2022, pp. 55–60, doi: [10.1145/3489517.3530409](https://doi.org/10.1145/3489517.3530409).
- [38] H. Jun, H. Ye, H. Jeong, and D. Chen, "AutoScaleDSE: A scalable design space exploration engine for high-level synthesis," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 16, no. 3, pp. 1–30, Sep. 2023.
- [39] M. G. Tallada, J. Denny, P. Valero-Lara, S. Lee, K. Teranishi, and J. S. Vetter, "ECC++: A compiler construction framework for embedded domain-specific languages," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. Workshops (IPDPSW)*, May 2024, pp. 667–677.
- [40] N. B. Agostini, S. Curzel, V. Amaty, C. Tan, M. Minutoli, V. G. Castellana, J. Manzano, D. Kaeli, and A. Tumeo, "An MLIR-based compiler flow for system-level design and hardware acceleration," in *Proc. 41st IEEE/ACM Int. Conf. Comput.-Aided Design*, Oct. 2022, pp. 1–9.
- [41] M. Nouar. *Understanding the Mathematics of PPO in Reinforcement Learning*. Accessed: Apr. 29, 2025. [Online]. Available: <https://medium.com/data-science/understanding-the-mathematics-of-ppo-in-reinforcement-learning-467618b2f8d4>
- [42] Medium. (2025). *Ppo: Efficient, Stable, and Scalable Policy Optimization*. Accessed: Apr. 29, 2025. [Online]. Available: <https://medium.com/@kdk199604/ppo-efficient-stable-and-scalable-policy-optimization-15b5b9c74a88>
- [43] Microchip. (2023). *SmartHLS High-Level Synthesis Software*. Accessed: Feb. 28, 2024. [Online]. Available: <https://www.microchip.com/en-us/products/fpgas-and-plds/fpga-and-soc-design-tools/smarthls-compiler>
- [44] GCC-GNU. (2023). *Other Built-in Functions Provided by GCC*. Accessed: Dec. 21, 2024. [Online]. Available: <https://gcc.gnu.org/onlinedocs/gcc/Other-Builtins.html>
- [45] J. Zhao, L. Feng, S. Sinha, W. Zhang, Y. Liang, and B. He, "Performance modeling and directives optimization for high-level synthesis on FPGA," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 7, pp. 1428–1441, Jul. 2020.
- [46] G. Jo, H. Kim, J. Lee, and J. Lee, "SOFF: An OpenCL high-level synthesis framework for FPGAs," in *Proc. ACM/IEEE 47th Annu. Int. Symp. Comput. Archit. (ISCA)*, May 2020, pp. 295–308.
- [47] H. Ye, H. Jun, J. Yang, and D. Chen, "High-level synthesis for domain specific computing," in *Proc. Int. Symp. Phys. Design*, Mar. 2023, pp. 211–219.
- [48] N. Wu, H. Yang, Y. Xie, P. Li, and C. Hao, "High-level synthesis performance prediction using GNNs: Benchmarking, modeling, and advancing," in *Proc. 59th ACM/IEEE Design Autom. Conf.*, Jul. 2022, pp. 49–54.
- [49] R. Zhao, J. Cheng, W. Luk, and G. A. Constantinides, "POLSCA: Polyhedral high-level synthesis with compiler transformations," in *Proc. 32nd Int. Conf. Field-Programmable Log. Appl. (FPL)*, Aug. 2022, pp. 235–242.
- [50] Y. Herklotz, J. D. Pollard, N. Ramanathan, and J. Wickerson, "Formal verification of high-level synthesis," *Proc. ACM Program. Lang.*, vol. 5, no. OOPSLA, pp. 1–30, Oct. 2021.
- [51] T. Ben-Nun, B. Ates, A. Calotoiu, and T. Hoefler, "Bridging control-centric and data-centric optimization," in *Proc. 21st ACM/IEEE Int. Symp. Code Gener. Optim.*, Feb. 2023, pp. 173–185.
- [52] K. Krommydas, W.-C. Feng, C. D. Antonopoulos, and N. Bellas, "OpenDwarfs: Characterization of dwarf-based benchmarks on fixed and reconfigurable architectures," *J. Signal Process. Syst.*, vol. 85, no. 3, pp. 373–392, Dec. 2016.
- [53] Berkley. (2023). *OpenDwarfs Benchmarks*. Accessed: Mar. 27, 2024. [Online]. Available: <https://github.com/vtsynergy/OpenDwarfs>
- [54] LLVM-MCA. (2023). *LLVM's Machine Code Analyzer*. Accessed: May 20, 2024. [Online]. Available: <https://llvm.org/docs/CommandGuide/llvm-mca.html>
- [55] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [56] Intel. (2024). *Intel Rendering Framework Using Software-Defined Visualization*. Accessed: Feb. 14, 2024. [Online]. Available: <https://www.intel.com/content/dam/develop/public/us/en/documents/parallel-universe-issue-35.pdf>



HAFAH SHAHZAD (Graduate Student Member, IEEE) received the bachelor's degree in electrical engineering from the School of Science and Engineering, LUMS, Pakistan, and the master's degree in power engineering from Technical University Munich, Germany. She is currently pursuing the Ph.D. degree with Boston University, with a focus on machine learning for compiler optimization and the different facets of cloud FPGAs, including architectures and increasing performance, and portability and programmability from compilers. Before joining the laboratory, she was a Validation Engineer at Texas Instruments, Germany.



ULRICH DREPPER is currently a Distinguished Engineer with Red Hat Research. He is also a part of the Office of the Chief Technical Officer and concentrates on developing new technologies for high-performance computing (and machine learning specifically), mostly in collaboration with university groups.



AHMED SANALLAH is currently a Principal Research Software Engineer with Red Hat. His current research interests include building open-source tooling for FPGAs that enables developers to easily and efficiently create custom hardware solutions, regardless of prior hardware development expertise.



SANJAY ARORA is currently a Research Scientist with Red Hat Research. He is also a part of the AI Center of Excellence, Red Hat. His research interests include machine learning, research, and the application of AI to systems.



MARTIN C. HERBORDT is currently a Professor with the Department of Electrical and Computer Engineering, Boston University, where he directs the Computer Architecture and Automated Design Laboratory.

...